Smart College Management System - Deep Project Flow

(Smart College Management System - Complete Project Flow Deep Dive)

1. Project overview / Project ka overview

English:

Smart College Management System is a full-stack college/school ERP-like web application. It connects the React frontend, FastAPI backend, SQL database, and Python face-recognition service. It supports admin, accountant, teacher, student, and parent roles. It manages academic, finance, attendance, communication, and settings modules.

Roman Urdu:

Smart College Management System ek full-stack college/school ERP-like web application hai. Yeh React frontend, FastAPI backend, SQL database, aur Python face-recognition service ko connect karta hai. Yeh admin, accountant, teacher, student, aur parent roles support karta hai. Yeh academic, finance, attendance, communication, aur settings modules manage karta hai.

2. High-level architecture / Architecture (overall structure)

English:

1. Frontend (React + Vite): pages, forms, API calls, theme, and UI state.

2. Backend (FastAPI): routers, business logic, validation, permissions, reports.

3. Database (SQLite/MySQL via SQLAlchemy): persistent data.

4. Face service (FastAPI + OpenCV): detect, embed, and match faces.

Main communication:

- Frontend calls backend REST APIs at `/api/*`.

- Backend checks auth token, runs logic, uses database.

- Face endpoints proxy to face service for ML operations.

- Settings and theme are stored in DB and applied to frontend.

Roman Urdu:

1. Frontend (React + Vite): pages, forms, API calls, theme, UI state.

2. Backend (FastAPI): routers, business logic, validation, permissions, reports.

3. Database (SQLite/MySQL via SQLAlchemy): data persistence.

4. Face service (FastAPI + OpenCV): faces detect, embed, match karna.

Main communication:

- Frontend backend REST APIs `/api/*` call karta hai.

- Backend auth token check karta hai, logic run karta hai, database use karta hai.

- Face endpoints face service ko ML operations ke liye proxy karte hain.

- Settings aur theme DB mein save hote hain aur frontend pe apply hote hain.

3. Authentication flow / Login flow

English:

1. User enters email/password on Login page.

2. Frontend calls `POST /api/login`.

3. Backend verifies password with bcrypt.

4. Backend creates JWT token with user id.

5. Frontend stores token in localStorage.

6. Frontend calls `/api/me` to load profile, permissions, branding, and locale.

7. App applies theme and routes by role (admin/dashboard; student/parent -> portal).

8. Language change calls `PUT /api/me` with `{ locale }`.

Roman Urdu:

1. User login page pe email/password enter karta hai.

2. Frontend `POST /api/login` call karta hai.

3. Backend password bcrypt se verify karta hai.

4. Backend user id ka JWT token create karta hai.

5. Frontend token localStorage mein save karta hai.

6. Frontend `/api/me` call karke profile, permissions, branding, locale load karta hai.

7. App theme apply karta hai aur role ke hisab se route karta hai (admin/dashboard; student/parent -> portal).

8. Language change `PUT /api/me` `{ locale }` call karta hai.

4. Core modules and their data flow / Modules ka flow

4.1 Students

English:

- List: `GET /api/students` with filters (search, class, section, gender, status, session, face).

- Create: form-data to `POST /api/students`; backend creates User (role student), Guardian, Student record, photo.

- View: `GET /api/students/{id}` returns full student with fees, attendance, marks, leaves.

- Edit: `PUT /api/students/{id}` updates user/student/guardian/photo.

- Withdraw: marks student inactive; account cannot sign in.

- Profile tabs: overview, attendance, fees, exams, leaves, documents.

Roman Urdu:

- List: `GET /api/students` with filters (search, class, section, gender, status, session, face).

- Create: form-data `POST /api/students`; backend User (role student), Guardian, Student record, photo create karta hai.

- View: `GET /api/students/{id}` full student with fees, attendance, marks, leaves return karta hai.

- Edit: `PUT /api/students/{id}` user/student/guardian/photo update karta hai.

- Withdraw: student inactive mark hota hai; account login nahi kar sakta.

- Profile tabs: overview, attendance, fees, exams, leaves, documents.

4.2 Teachers

English:

- List/search: `GET /api/teachers`.

- Create: JSON to `POST /api/teachers`; creates User + Teacher + assignments.

- Edit: `PUT /api/teachers/{id}` updates user/teacher fields and assignments.

- Status: active/inactive toggle.

- Face: admin can open registration per teacher.

Roman Urdu:

- List/search: `GET /api/teachers`.

- Create: JSON `POST /api/teachers`; User + Teacher + assignments create hote hain.

- Edit: `PUT /api/teachers/{id}` user/teacher fields aur assignments update karta hai.

- Status: active/inactive toggle.

- Face: admin per teacher face registration open kar sakta hai.

4.3 Classes, sections, subjects, sessions

English:

Structure endpoints manage academic structure. `/api/structure` returns classes with nested sections plus sessions and subjects. Frontend uses this to populate dropdowns.

Roman Urdu:

Structure endpoints academic structure manage karte hain. `/api/structure` classes with nested sections plus sessions aur subjects return karta hai. Frontend dropdowns populate karne ke liye use karta hai.

4.4 Attendance

English:

- Manual marking: `POST /api/attendance/manual`.

- Lists by person type/date/status/class/section.

- Close day: unmarked active students become Absent, or Leave if approved leave covers date.

- Face attendance scan: `/api/face/scan` matches face and marks attendance as present/late based on present-until time.

- Summary per student and month views.

- Offline sync supports original capture time and client id for deduplication.

Roman Urdu:

- Manual marking: `POST /api/attendance/manual`.

- Lists by person type/date/status/class/section.

- Close day: unmarked active students Absent ho jate hain, ya Leave if approved leave cover kare.

- Face attendance scan: `/api/face/scan` face match karta hai aur present-until time ke basis pe present/late mark karta hai.

- Summary per student aur month views.

- Offline sync original capture time aur client id deduplication support karta hai.

4.5 Fees

English:

- Fee structure/items per student: admission, monthly tuition.

- Class-wide assignment: `POST /api/fees/assign-class`.

- Payment collection: cash/bank/online/other; receipt number generated.

- Payment reversal available.

- Defaulter list and reminders.

- Pending fee reports.

Roman Urdu:

- Fee structure/items per student: admission, monthly tuition.

- Class-wide assignment: `POST /api/fees/assign-class`.

- Payment collection: cash/bank/online/other; receipt number generate hota hai.

- Payment reversal available.
- Defaulter list aur reminders.
- Pending fee reports.

4.6 Exams and results

English:

- Create exam for class/section/session.
- Date sheet/schedules with subject, date, times, room, teacher.
- Enter marks per student/subject.
- Publish/unpublish exam.
- Results calculate total, percentage, grade, pass/fail.
- Report card PDF.

Roman Urdu:

- Class/section/session ke liye exam create hota hai.
- Date sheet/schedules with subject, date, times, room, teacher.
- Marks per student/subject enter hote hain.
- Exam publish/unpublish hota hai.
- Results total, percentage, grade, pass/fail calculate karte hain.
- Report card PDF.

4.7 Leaves

English:

- Student/parent/teacher can apply for leave.
- Admin/approver approves or rejects.
- Approved leave counts toward student leave allowance.
- Approved leave on closed day is marked Leave, not Absent.

Roman Urdu:

- Student/parent/teacher leave apply kar sakte hain.
- Admin/approver approve ya reject karta hai.
- Approved leave student leave allowance mein count hota hai.
- Closed attendance day pe approved leave Leave mark hota hai, Absent nahi.

4.8 Academics: timetable, homework, assignments, announcements

English:

- Timetable: periods by day/subject/teacher/room.
- Homework: post per class/section with due date/file.
- Assignments: post with file; students submit work from portal.
- Announcements: audience all/class/parents/teachers; published/unpublished.

Roman Urdu:

- Timetable: periods by day/subject/teacher/room.
- Homework: class/section ke liye due date/file ke sath post hota hai.
- Assignments: file ke sath post; students portal se work submit karte hain.
- Announcements: audience all/class/parents/teachers; published/unpublished.

4.9 Notifications

English:

- User-specific notices with read/unread tracking.
- Mark read, mark all read, delete.
- Email/push/SMS/WhatsApp flags stored in settings.

Roman Urdu:

- User-specific notices with read/unread tracking.
- Read, all read, delete.
- Email/push/SMS/WhatsApp flags settings mein stored hote hain.

4.10 Reports

English:

- Filter by type and date; returns rows + CSV/PDF export.

Roman Urdu:

- Type aur date se filter; rows + CSV/PDF export return karta hai.

4.11 Settings

English:

- General, appearance (theme/colors/button shape), academic links, attendance, fees, notifications, language, security, users, backups, audit.

- Theme and colors apply via branding API.

Roman Urdu:

- General, appearance (theme/colors/button shape), academic links, attendance, fees, notifications, language, security, users, backups, audit.

- Theme aur colors branding API ke through apply hote hain.

4.12 Profile

English:

- Every user sees name/email/role/phone and photo.

- Students/teachers see registered face photo.

- Edit name/phone/photo/password.

Roman Urdu:

- Har user name/email/role/phone aur photo dekh sakta hai.

- Students/teachers registered face photo dekh sakte hain.

- Name/phone/photo/password edit kar saktay hain.

5. Face registration flow (detailed)

English:

1. Admin opens `/face/register`, chooses student/teacher.

2. Camera captures 5 views.

3. Movement check ensures not a still photo; one person only.

4. Press Save.

5. Backend calls face service `/embed-samples`.

6. Duplicate check compares new embedding with all other saved faces.

7. If duplicate, registration rejected with existing person's name.

8. If unique, backend stores embedding and preview photo in FaceTemplate.

9. Only admin can replace an existing face.

Roman Urdu:

1. Admin `/face/register` open karta hai, student/teacher choose karta hai.

2. Camera 5 views capture karta hai.

3. Movement check ensure karta hai photo still nahi hai; ek person only.

4. Save press karta hai.

5. Backend face service `/embed-samples` call karta hai.

6. Duplicate check new embedding ko all saved faces se compare karta hai.

7. Duplicate hone pe registration reject hota hai aur existing person ka naam show hota hai.

8. Unique hone pe backend embedding aur preview photo FaceTemplate mein store karta hai.

9. Existing face replace sirf admin kar sakta hai.

6. Face attendance scan flow (detailed)

English:

1. Open `/attendance/scan`.

2. Scanner calls `/api/face/health`.

3. Camera frame sent to `/api/face/look` to check face present.

4. When face detected, `/api/face/scan` calls `/match` against all saved embeddings.

5. If match score >= 0.55, backend marks attendance as present/late using present-until time.

6. If already scanned same day/client, response says already marked.

7. If no match, scan is rejected; wrong person cannot mark attendance for someone else.

8. Offline scans remain in browser and sync via `/api/face/sync`.

Roman Urdu:

1. `/attendance/scan` open hota hai.

2. Scanner `/api/face/health` call karta hai.

3. Camera frame `/api/face/look` pe sent hota hai face presence check ke liye.

4. Face detected hone pe `/api/face/scan` all saved embeddings ke against `/match` call karta hai.

5. Match score >= 0.55 ho to backend attendance present/late mark karta hai present-until time ke basis pe.

6. Agar same day/client already scan hua ho to response "already marked" aata hai.

7. No match hone pe scan reject hota hai; wrong person someone else ka attendance mark nahi kar sakta.

8. Offline scans browser mein rehte hain aur `/api/face/sync` se sync hote hain.

7. Permission and role flow

English:

- Backend permission service defines role permissions.

- Frontend `can(user, module)` checks permissions.

- Admin gets all major modules including face.

- Teacher gets academics/attendance/leaves etc.
- Accountant gets fees/reports/notifications.
- Student/parent get portal + notifications.

Roman Urdu:

- Backend permission service role permissions define karta hai.
- Frontend `can(user, module)` permissions check karta hai.
- Admin ko major modules including face milte hain.
- Teacher ko academics/attendance/leaves etc milte hain.
- Accountant ko fees/reports/notifications milte hain.
- Student/parent ko portal + notifications milte hain.

8. Database relationship summary

English:

- User: one student/teacher/guardian.
- Student: class, section, session, guardian, fees, payments, marks, leaves.
- Teacher: user, assignments (class/section/subject), face template.
- Attendance: polymorphic person_type/person_id.
- FaceTemplate: polymorphic person_type/person_id.
- FeeItem: student; Payment: student + fee item.
- Exam: class/section/session; schedules; marks.
- Leave: student/teacher.
- Announcement/AuditLog/Settings/Backup system tables.

Roman Urdu:

- User: ek student/teacher/guardian.
- Student: class, section, session, guardian, fees, payments, marks, leaves.
- Teacher: user, assignments (class/section/subject), face template.
- Attendance: polymorphic person_type/person_id.
- FaceTemplate: polymorphic person_type/person_id.
- FeeItem: student; Payment: student + fee item.
- Exam: class/section/session; schedules; marks.
- Leave: student/teacher.
- Announcement/AuditLog/Settings/Backup system tables.

9. Deployment and backup flow

English:

- Frontend built with `npm run build`.
- Backend run behind Nginx/PHP-FPM or uvicorn.
- MySQL for production; SQLite for local.
- Face service runs separately.
- Settings backups stored under backend storage.
- Audit logs track who changed what.

Roman Urdu:

- Frontend `npm run build` se build hota hai.
- Backend Nginx/PHP-FPM ya uvicorn ke behind run hota hai.
- Production MySQL; local SQLite.
- Face service separately run hota hai.
- Settings backups backend storage ke under save hote hain.
- Audit logs track karte hain kaun ne kya change kiya.

10. Complete project flow in one paragraph

English:

The user logs in, the backend returns a JWT and profile; frontend loads the dashboard or portal based on role. Admin/teachers manage students, fees, exams, attendance, and settings. Face registration saves unique face embeddings. Face attendance compares live face against saved faces, marks attendance, and supports offline sync. Students and parents use the portal for personal records. Settings apply theme/colors/language across the app. All data is stored in the database and reports can be exported as CSV/PDF.

Roman Urdu:

User login karta hai, backend JWT aur profile return karta hai; frontend role ke basis pe dashboard ya portal load karta hai. Admin/teachers students, fees, exams, attendance, settings manage karte hain. Face registration unique face embeddings save karta hai. Face attendance live face ko saved faces se compare karta hai, attendance mark karta hai, aur offline sync support karta hai. Students aur parents portal use karte hain personal records ke liye. Settings theme/colors/language across app apply karte hain. Sab data database mein store hota hai aur reports CSV/PDF export ho sakti hain.